QUY ƯỚC LÀM VIỆC NHÓM & PHÂN CÔNG ĐỒ ÁN

Đề tài: Configure and Deploy MLOps (Machine Learning Ops) Services on Arch Linux & Ubuntu

Môn học: Linux và Phần mềm Mã nguồn mở

Nhóm: 3 người

Tech Stack: Bash (IaC), Docker / Rootless Docker, Docker Compose, MLflow, PostgreSQL (Metadata Store), MinIO (S3 Object Storage), FastAPI (Inference Microservice), Prometheus + Grafana (Monitoring & Telemetry), nftables (Arch Linux), UFW (Ubuntu Server).

---

1. PHÂN CÔNG NHIỆM VỤ (3 THÀNH VIÊN)

🧑‍💻 Thành viên 1: Toàn quy n Node Arch Linux (Rolling Release & Hardening)

• **Trọng tâm:** Triển khai độc lập 100% Node Arch Linux, bảo mật hệ thống chuyên sâu (The Arch Way), Rootless Docker và MLOps Stack.

• **Chi tiết nhiệm vụ:**

1. **Hệ thống & Bảo mật (Arch Base):**

• Thiết lập user độc lập `mlops` (tách biệt hoàn toàn tài khoản cá nhân).

• Hardening OpenSSH: Port `2222`, vô hiệu hóa đăng nhập `root`, giới hạn số lần xác thực (`MaxAuthTries 3`).

• Tường lửa **`nftables`** stateful: Tích hợp rate-limiting chống brute-force SSH (tối đa 4 kết nối/phút), drop invalid packets, mở các port dịch vụ (`2222`, `8000`, `5000`, `3000`, `9001`).

• Tinh chỉnh Linux Kernel qua `sysctl` (`fs.file-max`, `net.core.somaxconn`, `net.ipv4.tcp_tw_reuse`).

• Viết script cài đặt (`01_system_hardening.sh`) và rollback an toàn (`01_rollback_hardening.sh`).

2. **Rootless Docker Engine:**

• Cấu hình User Namespaces (`/etc/subuid`, `/etc/subgid` với ánh xạ `mlops:100000:65536`).

• Cấu hình cgroups v2 resource delegation cho unprivileged user session.

• Kích hoạt `loginctl enable-linger mlops` để container chạy ngầm khi logout.

3. **MLOps Core Stack (Docker Compose):**

• Thiết kế `docker-compose.yml` phân tách 2 mạng Docker:

• `mlops-backend` (internal: true): Cô lập PostgreSQL (port 5432) và MinIO S3 API (port 9000), tuyệt đối không mở ra host IP.

• `mlops-frontend`: Mở các cổng dịch vụ công khai ra host (MLflow 5000, FastAPI 8000, Grafana 3000, MinIO Console 9001).

• FastAPI Inference Microservice: Đóng gói Dockerfile Multi-stage build chạy bằng user non-root (`USER 10001`), tích hợp Prometheus metrics `/metrics`.

4. **Systemd Lifecycle & Verification:**

• Viết User Systemd Service (`mlops-stack.service`) tự động khởi động stack khi boot.

• Viết script kiểm tra tự động (`04_verify_system.sh`) đánh giá tính toàn vẹn hệ thống và

bảo mật cổng.

🧑‍💻 Thành viên 2: Node Ubuntu — H t ng & B o m t (Infrastructure & Security)

• **Trọng tâm:** Triển khai nền tảng hạ tầng, bảo mật hệ điều hành Ubuntu Server (22.04/24.04 LTS), cài đặt Docker chuẩn Debian và quản trị dịch vụ hệ thống.

• **Chi tiết nhiệm vụ:**

1. **Hệ thống & Bảo mật (Ubuntu Base):**

• Thiết lập user deploy `mlops`.

• Hardening SSH Daemon: Đổi port `2222`, cấm đăng nhập tài khoản `root`.

• Tường lửa **`ufw`** chuẩn Ubuntu: Cấu hình mặc định deny incoming, mở đúng các port quy định (`2222`, `8000`, `5000`, `3000`, `9001`).

• Viết script cài đặt (`ubuntu/scripts/01_system_hardening.sh`) và rollback (`01_rollback_hardening.sh`).

2. **Docker Engine & Daemon Management:**

• Script tự động cài đặt Docker Engine và Docker Compose plugin chính thức qua kho APT (`download.docker.com`).

• Cấu hình Docker daemon cấp hệ thống qua systemd root (`systemctl enable --now docker`).

• Cấu hình log rotation cho Docker daemon để tránh đầy ổ cứng host.

3. **Triển khai & Kiểm thử Stack trên Ubuntu:**

• Chạy toàn bộ stack Docker Compose trên Ubuntu Server.

• Kiểm tra tính toàn vẹn của mạng nội bộ và các endpoint dịch vụ trên Ubuntu.

🧑‍💻 Thành viên 3: Node Ubuntu — Giám sát & Đo l ng So sánh (Observability & Benchmarking)

• **Trọng tâm:** Giám sát hệ thống (Prometheus + Grafana), cấu hình kết nối MLOps services và thực hiện kịch bản đo lường Benchmark so sánh giữa 2 hệ điều hành Linux.

• **Chi tiết nhiệm vụ:**

1. **Hệ thống Giám sát Hệ điều hành & Ứng dụng (Observability):**

• Cấu hình Prometheus (`prometheus.yml`): Cào metrics tài nguyên phần cứng từ Node Exporter và ứng dụng từ FastAPI.

• Cấu hình cảnh báo Prometheus Alert Rules khi quá tải tài nguyên hoặc dịch vụ dừng.

• Provisioning Grafana Dashboard: Tự động nạp DataSource và thiết kế Dashboard theo dõi tài nguyên phần cứng (CPU, RAM, Disk I/O) cùng lưu lượng API (RPS, Latency p95/p99).

2. **Tích hợp & Khởi tạo MLOps Services:**

• Cấu hình kết nối giữa MLflow Server với PostgreSQL Backend Store và MinIO Artifact Store.

• Tự động khởi tạo S3 Bucket (`mlflow-artifacts`) bằng `mc` (MinIO Client).

3. **Benchmarking & Báo cáo So sánh Linux:**

• Viết và thực thi kịch bản load test tự động (`benchmark.sh`): Đo thông lượng (RPS) và độ trễ (Latency) của FastAPI khi chịu tải trên Arch Linux (Rootless Docker) so với Ubuntu Server (Root Docker).

• Đo lường tiêu thụ RAM tĩnh (idle sau khi boot hệ điều hành) và RAM động (khi toàn bộ stack hoạt động).

• Tổng hợp bảng số liệu so sánh giữa Rolling Release vs. LTS, Rootless vs. Rootful containerization vào báo cáo đồ án.

---

2. QUY ƯỚC GIT

2.1. Phân nhánh (Branching Strategy)

• `main`: Nhánh nộp bài / Production – **KHÔNG push trực tiếp**.

• `dev`: Nhánh tích hợp và test chung – **KHÔNG push trực tiếp**.

• `feat/arch-*`: Toàn bộ tính năng thuộc Node Arch Linux (Thành viên 1).

• `feat/ubuntu-infra-*`: Tính năng hạ tầng, hardening, docker trên Ubuntu (Thành viên 2).

• `feat/ubuntu-monitor-*`: Tính năng monitoring, prometheus, grafana, benchmark trên Ubuntu (Thành viên 3).

• `fix/*`: Sửa lỗi hệ thống.

• `docs/*`: Cập nhật tài liệu, slide và báo cáo.

2.2. Chuẩn Commit Message (Conventional Commits)

• `feat(arch)`: Script và cấu hình cho Arch Linux.

• `feat(ubuntu-infra)`: Script hardening, ufw, cài đặt Docker trên Ubuntu.

• `feat(ubuntu-monitor)`: Script prometheus, grafana dashboards, benchmark.

• `fix(security)`: Sửa lỗi bảo mật, tường lửa, SSH.

• `docs(report)`: Cập nhật tài liệu báo cáo và số liệu đối sánh.

Ví dụ:

```bash
feat(arch): configure rootless docker namespaces and lingering
feat(ubuntu-infra): implement automated ufw rules and unattended docker install
feat(ubuntu-monitor): add prometheus scrape config and grafana dashboards
feat(benchmark): add rps and latency load test script
docs(report): update comparison table between arch and ubuntu
```

2.3. Quy tắc làm việc trên Git

• **Quy trình Pull Request (PR):**

1. Tạo nhánh từ `dev`: `git checkout dev && git pull && git checkout -b feat/ten-nhanh`
2. Viết code, kiểm tra chạy thử đạt yêu cầu.
3. Push lên remote: `git push origin feat/ten-nhanh`
4. Mở PR vào `dev` (mỗi PR không quá 500 dòng code).
5. Có ít nhất 1 thành viên khác review và Approve -> Merge.
6. Xóa branch sau khi merge hoàn tất.

• **Quy tắc an toàn:**

• **Tuyệt đối không commit file `.env` chứa mật khẩu thật lên Git.**
• Không dùng `git push --force` lên nhánh `dev` và `main`.
• Luôn pull nhánh `dev` về máy vào đầu mỗi phiên làm việc.

---

3. TIÊU CHUẨN KỸ THUẬT & CODE STYLE

3.1. Bash Script (Infrastructure as Code)

• Luôn khai báo `set -euo pipefail` ở đầu mọi script để bắt lỗi lập tức.
• Có trap hiển thị dòng code và câu lệnh lỗi (`trap '...' ERR`).
• Đảm bảo tính **Idempotent**: Chạy lại nhiều lần không sinh lỗi hoặc ghi trùng config.
• Bắt buộc có script **Rollback** tương ứng (`01_rollback_*.sh`) cho mọi script thay đổi hệ thống.
• Không viết comment dư thừa trong script; ưu tiên đặt tên biến rõ nghĩa và in log màu sắc (`log_info`, `log_success`, `log_error`).

3.2. Container & Kiến trúc MLOps

• **Kiến trúc Zero Trust:**
• Cổng 5432 (PostgreSQL) và Cổng 9000 (MinIO S3 API) nằm trọn trong mạng nội bộ `mlops-backend`. Host không bind cổng ra `0.0.0.0`.
• Chỉ các cổng UI/API cần giao tiếp ngoài mới bind ra host: `8000` (FastAPI), `5000` (MLflow UI), `3000` (Grafana), `9001` (MinIO Web Console).
• **Multi-Stage Build:** Dockerfile FastAPI dùng multi-stage build để tối ưu dung lượng và chạy bằng tài khoản unprivileged (`USER 10001`).

---

4. QUẢN LÝ BIẾN MÔI TRƯỜNG (.env.example)

Chỉ commit file mẫu `.env.example`:

```bash
# PostgreSQL (Metadata Store cho MLflow)
POSTGRES_USER=mlops
POSTGRES_PASSWORD=secure_postgres_password_change_me
POSTGRES_DB=mlflow_db
POSTGRES_PORT=5432
# MinIO (S3 Object Storage cho MLflow Artifacts)
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=secure_minio_password_change_me
MINIO_API_PORT=9000
MINIO_CONSOLE_PORT=9001
MLFLOW_BUCKET_NAME=mlflow-artifacts
# MLflow Tracking Server UI
MLFLOW_PORT=5000
```

```
# FastAPI Serving Microservice
FASTAPI_PORT=8000
# Telemetry & Monitoring
PROMETHEUS_PORT=9090
GRAFANA_PORT=3000
GRAFANA_ADMIN_PASSWORD=admin
NODE_EXPORTER_PORT=9100
```